

BazaarAI — Full Project Plan

The Big Picture (What We're Building)

BazaarAI is a system with three jobs. First, it automatically collects price data every day from government sources and newspapers. Second, it stores everything in a clean database — both raw numbers and news context. Third, it exposes that data to an AI through an MCP server so someone can ask "আজকে পেঁয়াজের দাম কত?" and get a real, sourced answer.

Think of it like a newsroom with three departments: the reporters (collectors), the archive (database), and the editor who answers questions (MCP + LLM).

The Tech Stack — With Model Recommendations

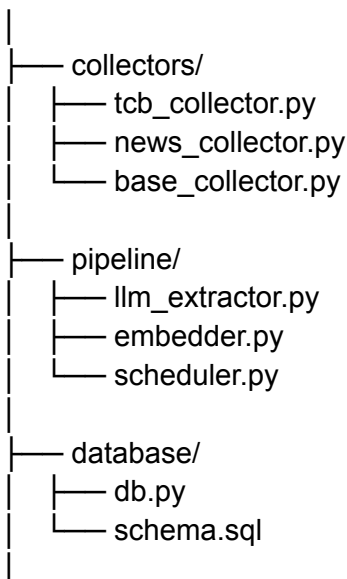
Before the folder structure, you need to understand what each tool does and *why* that specific tool for that specific job.

Layer	Tool	Why This One
Browser automation	Playwright	Can load JavaScript pages, bypass robots.txt restrictions, click buttons and download files — like a real human using a browser
HTML parsing	BeautifulSoup	Once Playwright loads a page, BeautifulSoup reads and picks out the exact pieces you want
Excel parsing	pandas + openpyxl	Already proven on your TCB file — reads Bengali text, handles merged cells
PDF parsing	pdfplumber	Best at reading text from tables inside PDFs, which is what DAM publishes
Scheduling	APScheduler	A Python library that says "run this function every day at 12:45 PM" — no separate app needed
Structured database	SQLite (Phase 1–2) → PostgreSQL (Phase 3+)	SQLite is a single file, zero setup, perfect for solo development. Migrate to PostgreSQL when you want multiple users or a cloud deployment

Vector database	ChromaDB (local mode)	Stores news article text as embeddings so the AI can search by meaning, not just keywords. Fully local, no cloud needed
Embeddings model	all-MiniLM-L6-v2 via sentence-transformers	A small, fast, free model that converts Bengali/English text into vectors. Runs entirely on your CPU — no GPU needed, fits your i5 machine
LLM for extraction	Groq — llama-3.1-8b-instant	Fast, free tier, perfect for structured extraction tasks like "read this article, give me prices as JSON." The 8B model is fast enough and accurate enough for this specific job
LLM for conversation	Groq — llama-3.3-70b-versatile	The bigger 70B model for the actual chatbot responses. Better at Bengali, better at reasoning, still free on Groq's generous tier
MCP server	FastAPI + official mcp Python library	FastAPI is the simplest way to build a Python API. The mcp library is Anthropic's open standard for connecting LLMs to tools
Dashboard	Streamlit	Pure Python, no frontend knowledge needed, looks clean out of the box

The Folder Structure — File by File

bazaardar/



```
|— mcp_server/
|   |— server.py
|   |— tools.py
|
|— dashboard/
|   |— app.py
|
|— downloads/      ← auto-created, gitignored
|— logs/           ← auto-created, gitignored
|— .env            ← API keys, never commit this
|— requirements.txt
|— README.md
```

collectors/ — The Reporters

base_collector.py This is a shared foundation that the other two collectors inherit from. It holds common logic that every collector needs — things like: how to start and close a Playwright browser session, how to save a downloaded file to the `downloads/` folder, how to write a log entry when something goes wrong, and a standard method for passing collected records to the database layer. You write this once and never repeat yourself in the other collector files.

tcb_collector.py This file's entire job is: open the TCB website, find today's Excel file, download it, parse it, return clean records. Specifically it does five things in sequence. First, it launches a Playwright browser (invisible/headless mode) and navigates to `tcb.gov.bd/pages/daily-rmps`. Second, it waits for the page's table to fully load — important because TCB's site is slow. Third, it finds the first row's `.xlsx` download link and clicks it, saving the file to `downloads/`. Fourth, it hands that file to pandas which reads rows starting from row 7, extracts item name, unit, today's min price, and today's max price — exactly like your current working parser. Fifth, it cleans the data (skips zero-price rows, skips section headers that have no unit) and returns a list of clean dictionaries ready to insert into the database. It also handles the "already downloaded today" case — if the file for today's date exists in `downloads/`, it skips the browser entirely and just re-parses the cached file.

news_collector.py This file handles all four newspapers — Jago News, Dhaka Post, Ittefaq, and Prothom Alo — because they all follow the same pattern. It contains a list of the four topic-page URLs at the top as a config variable. For each URL, it does the following: fetches the topic page using plain `requests` (no Playwright needed here since these pages render in basic HTML), finds all article links published in the last 24 hours by checking the date shown next to each headline, then for each article URL fetches the full article body text, strips out nav menus and ads using BeautifulSoup, and returns the clean Bengali article text alongside the article's

URL and publication date. This file does *not* extract prices itself — that's the pipeline layer's job. This file only collects raw text and passes it on.

pipeline/ — The Processing Engine

llm_extractor.py This file takes raw Bengali article text (from `news_collector.py`) and uses Groq's `llama-3.1-8b-instant` model to turn it into structured data. The reason we use the smaller 8B model here is that this is a mechanical extraction task — "find prices, output JSON" — not a creative or reasoning task. Speed matters more than depth because you might process 20 articles per day. The file sends each article to Groq with a carefully written system prompt that says something like: *"You are a data extraction tool. Read this Bengali news article and extract every food item price mentioned. Return only a JSON array. Each object must have: item_bn (Bengali name), item_en (English translation if clear, otherwise null), price_min, price_max, unit, market_name, and date. If a field is unclear, use null."* After getting the JSON back, this file validates it — checks that price values are actually numbers, skips any row where both min and max are null — and returns clean records. It also has a fallback: if Groq returns malformed JSON, it logs the raw response and moves on rather than crashing the whole pipeline.

embedder.py This file takes the same raw article text from the news collector and stores it in ChromaDB so the AI can search it later by meaning. The process is: take the full article text, split it into chunks of roughly 400 words with 50-word overlap (overlap ensures a sentence at the boundary of two chunks doesn't lose context), run each chunk through `sentence-transformers` using the `all-MiniLM-L6-v2` model to convert it to a vector (a list of 384 numbers representing the meaning), and store that vector in ChromaDB alongside metadata (source newspaper, article URL, publication date, the original chunk text). The overlap and chunking strategy is important — you want *"heavy rainfall destroyed chili crops in Kushtia"* and *"chili prices doubled in Dhaka"* to potentially be in the same or adjacent chunks so the AI connects them.

scheduler.py This is the alarm clock for the whole system. It uses APScheduler to define two daily jobs. Job 1 runs at 12:45 PM every day: runs `tcb_collector`, passes results to `db.py` for saving. Job 2 runs at 6:00 PM every day: runs `news_collector` for all four newspapers, passes each article through both `llm_extractor` (to get structured prices) and `embedder` (to store context in ChromaDB), saves structured prices to SQLite. It also runs a one-time startup check: if the database has zero records (first ever run), it triggers the TCB last-30-days collector immediately to backfill history. Logs every run's outcome — how many records saved, any errors — to `logs/scheduler.log`.

database/ — The Archive

schema.sql This file defines your two main tables in plain SQL. The **prices** table has columns for: id, date, item_bn (Bengali name), item_en (English name), unit, price_min, price_max, source (which website), source_type (either **official_tcb** or **news_reported**), market_name (e.g., Kawranbazar, Mirpur), and created_at timestamp. There's a unique constraint on **(date, item_bn, unit, source)** so the same item from the same source never gets duplicated even if the collector runs twice. The **articles** table has columns for: id, url, headline, source_name, published_date, raw_text, and processed (a boolean flag so the embedder knows which articles it has already vectorized). There's also a small **sources** table just to track which collector last ran successfully and when — useful for debugging.

db.py This is the single file that all other parts of the system talk to when they want to read or write data. It has simple functions: **save_prices(records_list)** which does a batch insert with duplicate-skipping, **get_prices_for_date(date)** which returns all items for a given day, **get_price_trend(item_name, days)** which returns a list of (date, price_min, price_max) tuples for charting, **save_article(url, text, date, source)** which stores a raw article, **mark_article_embedded(url)** which flips the processed flag after the embedder runs, and **get_unembedded_articles()** which returns articles the embedder hasn't processed yet. Using a single **db.py** as the gateway means if you ever switch from SQLite to PostgreSQL, you only change one file.

mcp_server/ — The Live Wire

tools.py This file defines what the AI is *allowed to ask your system*. Each function here is a tool the LLM can call. **get_today_prices(item_name=None)** — if item_name is given, returns prices just for that item from today; if null, returns all of today's prices. **get_price_trend(item_name, days=7)** — returns price history for the last N days so the AI can say "onion has been rising for 5 days." **search_news_context(query)** — takes the user's question, converts it to a vector using the same **sentence-transformers** model, searches ChromaDB for the 3 most semantically similar article chunks, and returns that text so the LLM can explain *why* prices are moving. **compare_sources(item_name)** — returns TCB official price vs newspaper-reported price side by side for the same item on the same date. **get_available_dates()** — tells the AI which dates have data, so it doesn't try to answer about dates with no records.

server.py This is the actual running server that wraps **tools.py** and exposes it through the MCP protocol. It starts a FastAPI application, registers each function from **tools.py** as an MCP tool with a short description of what it does and what parameters it accepts. That

description is what the LLM reads to decide which tool to call — so the descriptions need to be clear. It also handles authentication (a simple API key in the `.env` file so random people can't query your data), error responses (if a tool fails, return a clean error message rather than a Python stack trace), and logging of every tool call so you can see what the AI is asking for. When Claude or any MCP-compatible client connects to this server, it can see the available tools and call them mid-conversation.

dashboard/ — The Face

app.py A Streamlit application with three sections. The top section is a live price ticker — it queries today's prices from `db.py`, formats them as "সয়াবিন তেল: ৳185–195/L" and displays them in a scrolling horizontal line using Streamlit's `st.markdown` with a CSS animation. The middle section is a price comparison table — a dropdown to select an item, a date range slider, and a chart showing price trends over time using Plotly. It also shows the source breakdown (TCB official vs newspaper reported) as a color-coded column. The bottom section is a chat interface — a text input box where the user types a question in Bengali or English, the question goes to the MCP server which calls the appropriate tools, the 70B Groq model generates a response using both the tool results and the ChromaDB context, and the answer appears in the chat history above the input box. The chat remembers the last 5 exchanges so the user can ask follow-up questions naturally.

Phase Breakdown — What Gets Built When

Phase 1 — "Just get numbers in the database" (Week 1–2)

The goal here is purely mechanical. You want the system to wake up every day, download the TCB file, parse it, and save it without you doing anything. At the end of Phase 1 you should be able to open `prices.db` and see a week of clean price history.

What you build: `base_collector.py`, `tcb_collector.py`, `schema.sql`, `db.py`, and the TCB job in `scheduler.py`. You also run the TCB last-30-days URL once manually to backfill a month of history so your database isn't empty from day one.

What you test: Run the collector manually, confirm records appear in the database, confirm running it twice on the same day doesn't create duplicates. Then set the scheduler running and leave it for 3 days — if you see 3 new date entries without touching anything, Phase 1 is done.

What you do NOT build yet: anything AI-related, any UI, any news scraping.

Phase 2 — "Add newspaper context" (Week 3–4)

The goal here is to add the news layer. At the end of Phase 2 you should be able to query your database and see both structured prices from TCB and structured prices extracted by the LLM from newspaper articles, plus your ChromaDB should have article chunks stored and searchable.

What you build: `news_collector.py`, `llm_extractor.py`, `embedder.py`, and the news job in `scheduler.py`. You also write a small test script that takes one Jago News article, sends it to Groq, and prints what the LLM extracted — just to verify the extraction prompt is working before you wire it into the full pipeline.

The most important thing to get right in this phase is the Groq extraction prompt. Spend time testing it on 5–10 real articles and tuning the wording until the JSON output is consistently clean. A bad prompt here means corrupted data in your database.

What you test: Run the news collector on all four sources, confirm articles are saved to the `articles` table, confirm the LLM extractor is producing valid JSON and saving price records, confirm ChromaDB has entries you can query with a simple similarity search test.

Phase 3 — "Make the AI talk to the data" (Week 5–6)

The goal here is the MCP server. At the end of Phase 3 you should be able to type a question in Python (not even a UI yet) and get an AI response that uses real data from your database.

What you build: `tools.py`, `server.py`. You test it by connecting Claude Desktop to your local MCP server using the MCP config file and asking it questions directly in Claude's chat interface. This is the fastest way to validate the MCP tools without building a UI first.

The key thing to get right here is the tool descriptions in `server.py`. The LLM decides which tool to call based purely on reading those descriptions. If `get_today_prices` is described poorly, the LLM might call `search_news_context` instead and give a worse answer.

Phase 4 — "Make it beautiful and usable" (Week 7–8)

The goal here is the Streamlit dashboard. At the end of Phase 4 you have a complete, working application.

What you build: `dashboard/app.py` — the price ticker, comparison charts, and chat interface all wired together. You also write `requirements.txt` and a clean `README.md` so anyone can clone your repo and run it.

At this point BazaarDar is a complete V1 — daily automated collection from TCB and 4 newspapers, price storage with history, RAG-powered news context, MCP server exposing all data to an AI, and a dashboard with live chat. That is genuinely portfolio-grade work.

One Last Thing — `.env` File Structure

Every secret lives here and nowhere else. It holds your Groq API key, the MCP server's own API key (you generate this yourself, just a random string), the path to your SQLite database file, and a `DEBUG=true` flag you switch to `false` in production. This file is listed in `.gitignore` so it never accidentally gets pushed to GitHub.